

SymPy Bug Report

1 Bug 40: Even Powers of Complex Sign Collapse to One

1.1 Minimal Reproducer

```
from sympy import Abs, I, N, sign

z = 1 + I
actual = sign(z) ** 2
expected = z ** 2 / Abs(z) ** 2

print("z:", z)
print("actual:", actual)
print("expected:", expected)
print("numeric actual:", N(actual, 50))
print("numeric expected:", N(expected, 50))
```

1.2 Observed Behavior

[illegible]

SymPy evaluates `sign(1 + I)**2` to 1. This is a mathematically false simplification for the non-real complex argument $1 + i$: the complex sign is a unit complex direction, not a real-valued sign constrained to ± 1 .

1.3 Expected Behavior

For $z = 1 + i$, $\text{sign}(z)^2$ should evaluate consistently with the complex-sign definition:

$$\text{sign}(1+i)^2 = \frac{(1+i)^2}{\text{Abs}(1+i)^2} = \frac{(1+i)^2}{2} = i.$$

Equivalently, SymPy should leave the power unevaluated rather than returning 1 when it cannot justify the complex-direction value.

1.4 Mathematical Explanation

For a nonzero complex number z , SymPy's documented complex sign represents the unit direction

$$\text{sign}(z) = \frac{z}{|z|}.$$

Therefore,

$$\text{sign}(z)^2 = \frac{z^2}{|z|^2}.$$

This expression is not identically 1 over the non-real complex numbers. In the representative case,

$$\frac{(1+i)^2}{|1+i|^2} = \frac{1+2i+i^2}{2} = \frac{2i}{2} = i.$$

The returned value 1 is not an equivalent representation of i , not a harmless branch choice, and not a delayed symbolic form. It changes the complex direction by replacing the square of a unit complex number with the real-sign identity $\text{sign}(x)^2 = 1$, which is valid for nonzero real x but not for arbitrary nonzero complex z .

1.5 Source-Level Diagnosis

The source-level diagnosis identifies `sympy/functions/elementary/complexes.py`, specifically the method `sign._eval_power`, as the immediate cause. The traced call path is: `sign(1 + I)` remains unevaluated in `sign.eval` because the argument is neither real nor purely imaginary; constructing `sign(1 + I)**2` then dispatches to `sign._eval_power`. That method applies an even-power shortcut using only a nonzero-argument check and an even-integer exponent check:

```
if (
    fuzzy_not(self.args[0].is_zero) and
    other.is_integer and
    other.is_even
):
    return S.One
```

For the argument $1 + i$, `is_zero` is false and the exponent 2 is an even integer, so the method returns `S.One` immediately. This bypasses the complex-sign definition $\text{sign}(z) = z / \text{Abs}(z)$, which would give i after squaring. The diagnosis therefore indicates that the shortcut is overbroad: it applies a real-valued identity to an unevaluated non-real complex sign. A plausible fix direction supported by the diagnosis is to restrict this shortcut to cases where the sign value is known to square to 1, such as real signs, or otherwise leave the power unevaluated or rewrite through $z / \text{Abs}(z)$ for general complex arguments.

1.6 Additional Failing Instances

The same failure occurs for a broader family of non-real complex constants. In each case below, SymPy collapses $\text{sign}(z)^2$ to 1, while the complex-sign definition gives $z^2 / \text{Abs}(z)^2$. Since each listed z is nonzero and non-real, the expected behavior follows from the same unit-direction identity rather than from a special property of the representative value $1 + i$.

Input / Expression	SymPy behavior	Expected behavior
$\text{sign}(1 + i)^2$	1	i
$\text{sign}(1 + 2i)^2$	1	$(-3 + 4i)/5$
$\text{sign}(2 + i)^2$	1	$(3 + 4i)/5$
$\text{sign}(-1 + i)^2$	1	$-i$
$\text{sign}(2 - i)^2$	1	$(3 - 4i)/5$
$\text{sign}(3/5 + i/7)^2$	1	$(208 + 105i)/233$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material found was SymPy’s elementary-functions documentation, which confirms that complex sign denotes a unit complex direction, and a Stack Overflow discussion about a `gruntz/limit` failure involving sign of a complex expression. Those are related to complex-sign handling in general, but they do not describe `sign(1 + I)**2` returning 1, the overbroad `sign._eval_power` shortcut, or this exact algebraic simplification failure. The deduplication verdict is a new instance of a known failure mode: complex-sign handling has related public precedent, but this exact reproducible even-power collapse appears previously unreported and remains individually actionable as a SymPy issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import Abs, I, N, Rational, sign

CASES = [
    1 + I,
    1 + 2 * I,
    2 + I,
    -1 + I,
    2 - I,
    Rational(3, 5) + I / 7,
]

@pytest.mark.parametrize("z", CASES)
def test_even_power_of_complex_sign_uses_complex_direction(z):
    actual = sign(z) ** 2
    expected = z ** 2 / Abs(z) ** 2

    assert abs(complex(N(actual, 80)) - complex(N(expected, 80))) < 1e-45
```